NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
(FAST-NUCES) KARACHI CAMPUS

PROJECT PROPOSAL

Database Systems (CS-3304)

UniCarpool

A University Ride-Sharing Web Application

Student Names	Muhammad Umer 24K-0894 Ahmed Khan 24K-0629 Muhammad Hammad Yousuf 24K-0597
Course	Database Systems (CS-3304)
Instructor	Mr. Talha Shahid
Department	Computer Science
Submission Date	10 May 2026

Abstract

UniCarpool is a full-stack web application designed to address the persistent commuting challenges confronted by students and faculty at Pakistani universities, with particular relevance to the diverse urban geography of Karachi. The platform enables registered users to post available ride offers, discover and join ongoing rides, manage multi-stop pickup routes, and settle fares through a simulated payment workflow — all within a secure, session-authenticated environment.

The system is architected on Django 5.2, Python's most mature and production-ready web framework, backed by a MySQL 8.x relational database. The core database schema comprises four interrelated tables: auth_user (leveraging Django's built-in user model), rides_ride, rides_ridepickupstop, and rides_rideparticipant. These tables are normalized to Boyce-Codd Normal Form (BCNF) and linked through cascading foreign key relationships that guarantee referential integrity throughout every stage of a ride's lifecycle.

Key technical contributions of the project include atomic seat-booking transactions using SELECT FOR UPDATE row-level locking to prevent race conditions under concurrent load, Django ORM query optimization via select_related and prefetch_related to eliminate N+1 query problems, and a stateful ride lifecycle management system spanning three states: OPEN, ACTIVE, and FINALIZED. The frontend layer delivers a mobile-responsive, dark-themed interface built with Bootstrap 5 and a custom CSS design system featuring glass-morphism panels and animated visual elements.

The project is deployable on any environment where Python 3.13 and MySQL 8 are available, with all configuration — including database credentials — driven by environment variables for portability and security. It serves simultaneously as a practical solution to a real-world campus mobility problem and as a rigorous academic exercise in relational database design, ORM-level query engineering, and full-stack web development.

Attribute	Details
Project Title	UniCarpool — University Ride-Sharing Web Application
Technology Stack	Django 5.2, MySQL 8.x, Bootstrap 5.3.3, PyMySQL 1.1.1, Python 3.13
Database	MySQL with Django ORM; 4 core tables, BCNF normalized
Key Features	User authentication, ride creation, seat booking, multi-stop pickup, ride finalization, payment simulation, ride history
Architecture	Django MVT pattern; InnoDB RDBMS backend; Bootstrap 5 responsive frontend
Team Members	Muhammad Umer (24K-0894), Ahmed Khan (24K-0629), Muhammad Hammad Yousuf (24K-0597)

1. Introduction

1.1 Background

Transportation is one of the most persistent day-to-day challenges for university students in large Pakistani urban centers. Students and faculty commuting across Karachi's sprawling geography — from areas such as Gulshan-e-Iqbal, Gulistan-e-Johar, Malir, and North Karachi to the FAST-NUCES main campus — frequently travel alone in private vehicles or depend on a public transport system widely regarded as inconsistent, overcrowded, and inadequately safe.

The concept of carpooling — coordinating shared vehicle use among individuals traveling in the same direction — represents a widely proven and practically effective mitigation for these challenges. Mature carpooling platforms such as BlaBlaCar (Europe), Zimride (US universities), and Careem Pool (MENA) have demonstrated measurable cost reduction, reduced urban traffic congestion, and lower per-capita carbon emissions. However, purpose-built carpooling platforms tailored to the social dynamics, trust requirements, and geographic patterns specific to South Asian universities remain comparatively underdeveloped.

UniCarpool was conceived to fill this gap. By building a closed, university-branded platform accessible exclusively to registered community members, it creates a trusted environment where drivers and passengers can coordinate shared commutes with verifiable identities, structured seat-availability tracking, and formal ride management — elements absent from informal coordination channels such as WhatsApp groups.

1.2 Problem Statement

University students at FAST-NUCES Karachi, like those at many large Pakistani universities, face significant commuting inefficiencies. Public transport schedules are inconsistent and routes frequently do not align with campus-to-residential commute patterns. Commercial ride-hailing platforms such as Careem and inDrive, while convenient, impose per-ride costs that can be prohibitive for students operating on limited monthly budgets. No centralized, university-scoped platform currently exists to allow students to coordinate shared rides based on mutual routes, scheduled departure times, and trusted university membership.

In the absence of such a system, students organize informal carpools through WhatsApp groups or social media posts — methods that are neither scalable nor systematically safe. These informal arrangements provide no structured seat-availability tracking, no agreed-upon fare management, no ride history or accountability layer, and no protection against double-booking or ride cancellation disputes.

1.3 Motivation

The primary motivation for developing UniCarpool lies in delivering tangible value to the university community. Drivers can offset recurring fuel and vehicle maintenance costs through fare-sharing. Passengers gain door-to-door or multi-stop convenience at a fraction of commercial platform prices. Environmental impact is reduced through vehicle consolidation. The broader campus community benefits from a safer, more organized commuting culture reinforced by a digital platform.

From an academic perspective, the project provides an ideal vehicle for demonstrating competencies in relational database design, web application security, ORM-level query engineering, and scalable backend architecture — all core expectations for graduating computer science students.

1.4 Objectives

The project pursues the following primary objectives:

- Design and implement a fully normalized relational database schema (BCNF-compliant) for a carpool management system using MySQL 8.x and Django ORM.
- Develop a secure, session-based user authentication system leveraging Django's built-in contrib.auth framework with PBKDF2-SHA256 password hashing.
- Build a ride-posting module enabling drivers to specify origin, destination, departure time, fare per seat, total capacity, and an ordered sequence of pickup stops.
- Implement a concurrent-safe seat-booking system using atomic database transactions and SELECT FOR UPDATE row-level locking.
- Provide a driver-side ride finalization workflow that triggers the passenger payment simulation phase.
- Deliver a clean, fully responsive frontend using Bootstrap 5 and a custom dark-themed CSS design system compatible with mobile viewports from 375px.
- Demonstrate database integrity through foreign key constraints with cascading deletes, unique composite constraints, and indexed query fields.

1.5 Scope

UniCarpool covers the following functional scope: user registration, login, and logout; ride creation with full route and scheduling details; ride discovery filtered to available future rides posted by other users; ride joining with concurrency protection; driver-side ride finalization; passenger-side payment simulation; and ride history for both roles. The project explicitly excludes: real-time geolocation or map integration, push notification delivery systems, live payment gateway integration, in-app messaging, and native mobile applications — all of which are documented as planned future enhancements.

2. Literature Review

2.1 Relational Database Systems and MySQL

Relational database management systems (RDBMS) based on E.F. Codd's relational model remain the dominant paradigm for structured data management in enterprise and web applications. MySQL 8.x, the RDBMS selected for this project, is one of the world's most widely deployed open-source databases. The InnoDB storage engine — MySQL's default since version 5.5 — provides full ACID transaction support (Atomicity, Consistency, Isolation, Durability), row-level locking rather than table-level locking (critical for high-concurrency applications such as seat booking), foreign key constraint enforcement, and MVCC (Multi-Version Concurrency Control) for snapshot isolation. These features directly informed the transaction design in UniCarpool's seat-booking algorithm.

Database normalization theory, formalized by Codd (1970, 1971, 1974) and extended by Boyce and Codd (1974) to BCNF, underpins the schema design of this project. The progressive elimination of update anomalies, deletion anomalies, and insertion anomalies through normalization ensures long-term data consistency — a requirement that becomes increasingly important as the user base and ride volume grow.

2.2 Django as a Web Framework

Django is a high-level Python web framework following the Model-View-Template (MVT) architectural pattern. First released in 2005 and currently at version 5.2, Django is characterized by its 'batteries-included' philosophy: built-in components for authentication (django.contrib.auth), ORM-based database abstraction, form handling and validation, CSRF protection, session management, and URL routing significantly reduce development time and eliminate common security pitfalls. Django's ORM translates Python model class definitions into SQL DDL and generates parameterized SQL for all DML operations, effectively preventing SQL injection vulnerabilities in normal usage.

The select_related and prefetch_related ORM methods, both used extensively in UniCarpool's views, address the N+1 query problem — a common performance issue in ORM-backed applications where listing N items with related data generates N+1 database queries. select_related performs a SQL JOIN for ForeignKey relationships, while prefetch_related uses separate optimized queries with Python-level join for ManyToMany and reverse FK relationships.

2.3 Similar Existing Systems

Several existing systems share domain overlap with UniCarpool. BlaBlaCar is a commercial long-distance ride-sharing platform operating across 22 countries, featuring route matching, verified profiles, and integrated payment. Its design informed UniCarpool's fare-per-seat model and ride lifecycle concepts, though its scope (intercity travel between strangers) differs fundamentally from UniCarpool's university-community focus. Zimride (now Enterprise Rideshare), developed originally at Cornell University, represents the closest comparable academic platform — a closed university carpooling network with institutional email verification and route matching. Careem Pool, the local market leader in Pakistan, offers shared rides but operates as a commercial taxi platform at commercial rates, without any university-scoped membership model. GoRide and similar university informal apps lack the database integrity guarantees and authentication structures present in UniCarpool.

Feature	UniCarpool	BlaBlaCar	Careem Pool	WhatsApp Groups
University-Scoped	Yes	No	No	Informal
Structured DB	Yes (MySQL/BCNF)	Yes	Yes	No
Seat Tracking	Yes (atomic)	Yes	Yes	No
Multi-Stop Pickup	Yes	Limited	No	No
Payment Integration	Simulated	Yes (real)	Yes (real)	No
Open Source	Yes	No	No	N/A

3. System Requirements

3.1 Functional Requirements

3.1.1 User Authentication Module

- FR-01: The system shall allow new users to register with first name, last name, username, email, and password through a validated signup form.
- FR-02: The system shall authenticate users via session-based login using username and password credentials.
- FR-03: The system shall support secure logout, terminating the user session.
- FR-04: All ride-related pages shall require an authenticated user session; unauthenticated requests shall be redirected to the login page.

3.1.2 Ride Creation Module

- FR-05: An authenticated user (acting as driver) shall be able to create a new ride specifying: origin, destination, departure date and time, primary pickup point, ordered pickup stops (free-text, comma or newline-delimited), total seat capacity, fare per seat (decimal), and optional notes.
- FR-06: The system shall validate that the scheduled departure time is strictly in the future at the time of form submission.
- FR-07: The system shall parse and persist pickup stops as ordered RidePickupStop records linked to the created ride, deduplicating identical stop names.

3.1.3 Ride Discovery Module

- FR-08: Authenticated users shall see a paginated list of all available rides (status OPEN or ACTIVE, seats_available > 0, departure_time in future) posted by other users.
- FR-09: Each ride card shall display: driver username, origin, destination, departure time, pickup stops, fare per seat, and remaining seats.

3.1.4 Seat Booking Module

- FR-10: An authenticated user shall be able to join a ride by providing a contact number and preferred pickup location via an inline form.
- FR-11: The system shall prevent a user from joining their own ride.
- FR-12: The system shall prevent duplicate bookings (same user, same ride).
- FR-13: Seat decrement shall be performed atomically using database-level locking to prevent race conditions under concurrent requests.

3.1.5 Ride Management Module

- FR-14: Drivers shall be able to view all their active rides with a passenger list on the Current Rides dashboard.
- FR-15: A driver shall be able to finalize a ride, transitioning its status to FINALIZED and recording finalized_at timestamp.
- FR-16: Only the ride's driver shall be authorized to finalize it; other users' finalization attempts shall receive HTTP 404.

3.1.6 Payment Simulation Module

- FR-17: After a ride is finalized, participating passengers shall be able to simulate payment by clicking a confirmation button.
- FR-18: Payment simulation shall transition payment_status from PENDING to PAID and record paid_at timestamp.
- FR-19: Payment simulation shall only be permitted after ride finalization; attempts on non-finalized rides shall display an error.

3.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All passwords stored as PBKDF2-SHA256 hashes; plaintext passwords never persisted
NFR-02	Security	CSRF protection enforced on every POST form via Django CsrfViewMiddleware
NFR-03	Security	All database queries use parameterized SQL (ORM); no raw concatenated queries
NFR-04	Performance	Home page ride listing query executes in under 100ms with up to 500 active rides via select_related / prefetch_related
NFR-05	Reliability	Concurrent seat-booking requests for the last available seat shall result in exactly one successful booking
NFR-06	Usability	All pages render correctly on mobile viewports from 375px wide without horizontal scrolling
NFR-07	Portability	All configuration (DB credentials, secret key) loaded from environment variables; no hardcoded secrets
NFR-08	Maintainability	Database schema versioned via Django migrations; all changes reproducible from migrations/

3.3 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or better
RAM	4 GB	8 GB or more
Storage	20 GB HDD	50 GB SSD
Network	Broadband (1 Mbps)	10 Mbps or better
Client Device	Any device with a modern browser	Desktop or smartphone with Chrome/Firefox/Safari

3.4 Software Requirements

Software	Version	Purpose
Python	3.13+	Primary programming language
Django	5.2	Backend web framework (MVT, ORM, auth, forms)
MySQL	8.x	Relational DBMS (InnoDB engine)
PyMySQL	1.1.1+	Pure-Python MySQL database adapter
Bootstrap	5.3.3 (CDN)	Responsive frontend CSS framework
Git	Latest	Version control and collaboration
OS (Server)	Ubuntu 22.04 LTS	Production server operating system
Browser (Client)	Chrome 110+ / Firefox 110+	User-facing interface access

4. Feasibility Study

4.1 Technical Feasibility

The project leverages exclusively open-source, stable, and battle-tested technologies: Python 3.13, Django 5.2, and MySQL 8.x are all mature projects with substantial community support, extensive documentation, and long-term support commitments. All three team members possess working knowledge of Python, web frameworks, and relational database fundamentals from prior coursework. The development environment — local Python virtual environments with MySQL running via XAMPP or native MySQL Community Server — is readily available on standard student laptops. The absence of paid licenses, proprietary SDKs, or specialized hardware makes the project entirely feasible within university resource constraints.

4.2 Economic Feasibility

The total cost of development is effectively zero beyond developer time. All technologies (Python, Django, MySQL Community Edition, Bootstrap, Git, GitHub) are freely available under open-source licenses. Hosting costs are also zero during development, as the application runs on localhost. For production deployment, options such as Railway.app and Render.com offer free tiers sufficient for a university-scale application. No external API subscriptions or paid services are required for the functionality described in this proposal.

4.3 Operational Feasibility

The system targets a clearly defined user base — university students and faculty at FAST-NUCES Karachi — who are uniformly tech-savvy and accustomed to web-based platforms. The user interface is designed with a minimal learning curve: the core user journey (register, browse rides, join a ride) can be completed in under three minutes by a first-time user. Administrative overhead is minimal, with Django's built-in admin panel providing a full CRUD interface for database management without requiring custom admin development. The platform does not depend on any third-party service availability (e.g., payment gateways or map APIs in its current form), ensuring high operational reliability.

4.4 Schedule Feasibility

The project was completed over an approximately eight-week development cycle, spanning requirements analysis, database design, model and view implementation, frontend development, testing, and documentation. This timeline is consistent with the academic semester schedule and the available working hours of the three-member development team. A detailed project timeline is provided in Section 10.

5. Proposed System

5.1 System Overview

UniCarpool proposes a closed, university-branded ride-sharing platform accessible exclusively to registered community members. The system employs a three-tier architecture: a MySQL 8.x relational database at the persistence layer, Django 5.2 as the application server handling all business logic and ORM translation, and a Bootstrap 5 / custom CSS frontend rendered through Django's template engine. The platform enforces community trust by requiring account registration before any ride interaction. All sensitive operations — seat booking, ride finalization, payment simulation — are transactionally protected at the database level.

5.2 System Architecture

The overall system follows Django's MVT (Model-View-Template) pattern organized within two Python packages:

- config/ — the Django project package containing settings.py (database configuration, installed applications, middleware stack, static files configuration), urls.py (root URL dispatcher including admin, Django auth, and rides application routes), and WSGI/ASGI deployment interface modules.
- rides/ — the core application package containing models.py (ORM model class definitions), views.py (HTTP request handler functions), forms.py (input validation and ModelForm logic), urls.py (application-level URL patterns mapped to view functions), admin.py (Django admin site registrations), and tests.py (automated test suite using Django TestCase).

Client HTTP requests are received by the Django WSGI server (or Gunicorn in production). Django's URL dispatcher routes each request to the appropriate view function. Views interact with models through the ORM, which generates and executes parameterized SQL against the MySQL InnoDB database. Template rendering produces HTML responses delivered to the browser.

5.3 Main Modules

Module	Description
Authentication	User registration (SignUpForm), session-based login (StyledAuthenticationForm), logout. Delegates password hashing and session management to django.contrib.auth.
Ride Creation	RideCreateForm validates all ride fields. pickup_stops_text field parses free-text stop input into ordered RidePickupStop records. Future-departure validation enforced at the form layer.
Ride Discovery	home_view queries available future rides with select_related and prefetch_related, excluding the current user's own rides and rides already joined.
Seat Booking	join_ride_view executes within @transaction.atomic. Uses select_for_update() to acquire InnoDB row lock before seat decrement, preventing TOCTOU race conditions.

Module	Description
Ride Management	current_rides_view presents separate driver and passenger dashboards. finalize_ride_view transitions ride state to FINALIZED within an atomic transaction.
Payment Simulation	simulate_payment_view enforces a PENDING → PAID state machine transition, gated behind ride FINALIZED status verification.
Ride History	recent_rides_view presents completed rides for both driver and passenger roles, with pending payment counts prominently displayed.

5.4 User Interaction Workflow

The typical user interaction sequence unfolds as follows. A new user visits the landing page and registers an account. Upon login, they are presented with the home page listing available rides. A student wishing to drive posts a new ride via the Create Ride form, specifying route details, departure time, capacity, fare, and pickup stops. A student wishing to travel as a passenger browses available rides, selects a suitable option, and joins by providing a contact number and preferred pickup location. The driver views their active ride's passenger list on the Current Rides dashboard. At the conclusion of the physical commute, the driver finalizes the ride. Passengers then see the ride appear in their Recent Rides history with a payment simulation button, which they click to complete the settlement process.

6. Database Design

6.1 ER Diagram Description

The entity-relationship model for UniCarpool comprises four entities. The USER entity (mapped to Django's auth_user table) represents a registered platform user identified by a unique username and email. A user may occupy two concurrent roles: driver (creator of rides) and passenger (participant in rides posted by others). The RIDE entity represents a single carpool trip, owned by exactly one driver, characterized by origin, destination, departure schedule, seat capacity, fare, and lifecycle status. The RIDE_PICKUP_STOP entity represents an ordered intermediate pickup location along a ride's route, associated with exactly one ride, with a unique stop order per ride to enable ordered retrieval. The RIDE_PARTICIPANT entity represents a confirmed passenger booking, linking exactly one user to exactly one ride, capturing contact information, selected pickup location, and payment state.

Cardinality summary: USER (1) drives RIDE (many); USER (1) participates RIDE_PARTICIPANT (many); RIDE (1) has RIDE_PICKUP_STOP (many); RIDE (1) has RIDE_PARTICIPANT (many). All foreign key relationships implement ON DELETE CASCADE, ensuring that deletion of a user removes all their driven rides and participations, and deletion of a ride removes all its pickup stops and participant records.

6.2 Table Definitions

Table 6.1 — auth_user (Django Built-in)

Column	Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique user identifier
username	VARCHAR(150)	NOT NULL, UNIQUE	Login username
email	VARCHAR(254)	NOT NULL	User email address
password	VARCHAR(128)	NOT NULL	PBKDF2-SHA256 hashed password
date_joined	DATETIME	NOT NULL	Account creation timestamp

Table 6.2 — rides_ride

Column	Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique ride identifier
driver_id	BIGINT	FK → auth_user.id, CASCADE	Driving user
origin	VARCHAR(120)	NOT NULL	Starting location
destination	VARCHAR(120)	NOT NULL	Ending location

Column	Type	Constraints	Description
departure_time	DATETIME	NOT NULL, INDEX	Scheduled departure
seats_total	INT UNSIGNED	NOT NULL, DEFAULT 1	Total declared capacity
seats_available	INT UNSIGNED	NOT NULL, DEFAULT 1	Remaining joinable seats
fare_per_seat	DECIMAL(7,2)	NOT NULL	Cost per seat in PKR
status	VARCHAR(16)	NOT NULL, DEFAULT 'OPEN', INDEX	OPEN | ACTIVE | FINALIZED
finalized_at	DATETIME	NULL	Finalization timestamp
created_at	DATETIME	NOT NULL, DEFAULT NOW()	Record creation time
updated_at	DATETIME	NOT NULL, ON UPDATE NOW()	Last modification time

Table 6.3 — rides_ridepickupstop

Column	Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique stop identifier
ride_id	BIGINT	FK → rides_ride.id, CASCADE	Parent ride
location	VARCHAR(120)	NOT NULL	Stop location name
stop_order	INT UNSIGNED	NOT NULL, DEFAULT 1, UNIQUE(ride_id, stop_order)	Ordering sequence (1 = first stop)

Table 6.4 — rides_rideparticipant

Column	Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique participant record ID
ride_id	BIGINT	FK → rides_ride.id, CASCADE	The ride being joined
user_id	BIGINT	FK → auth_user.id, CASCADE,	Passenger user

Column	Type	Constraints	Description
		UNIQUE(ride_id, user_id)	
contact_number	VARCHAR(20)	NOT NULL	Passenger contact visible to driver
pickup_location	VARCHAR(120)	NOT NULL	Selected pickup stop
payment_status	VARCHAR(12)	NOT NULL, DEFAULT 'PENDING'	PENDING | PAID
paid_at	DATETIME	NULL	Payment simulation timestamp
joined_at	DATETIME	NOT NULL, DEFAULT NOW()	Booking creation timestamp

6.3 Normalization Analysis

The schema is fully normalized to Boyce-Codd Normal Form (BCNF). All tables satisfy First Normal Form (1NF): every column stores a single atomic value, there are no repeating groups, and each row is uniquely identified by a surrogate primary key. The multi-stop requirement was correctly extracted into a separate rides_ridepickupstop table rather than stored as a delimited string within rides_ride.notes, eliminating the repeating group that would otherwise violate 1NF.

Second Normal Form (2NF) is trivially satisfied because all custom tables use single-column surrogate primary keys, eliminating the possibility of partial key dependencies. Third Normal Form (3NF) holds throughout: in rides_ride, all non-key attributes depend solely on id and not transitively on each other. In rides_rideparticipant, contact_number and pickup_location depend on the specific booking identified by (ride_id, user_id), not on any derived attribute. BCNF is satisfied because each table contains exactly one candidate key (the surrogate PK), and every functional dependency is of the form {PK} → {non-key attribute}.

7. Implementation

7.1 Development Methodology

The project followed an Agile-inspired iterative development process organized into focused sprints, each delivering a demonstrable increment of functionality. The initial sprint established the project skeleton (Django project initialization, MySQL configuration, auth_user integration, and the base HTML template). Subsequent sprints implemented the Ride model and creation workflow, the ride discovery and home page, the seat-booking logic, the Current Rides dashboard, ride finalization, payment simulation, and the Recent Rides history view. Each sprint ended with a functional test of the delivered features before proceeding. Version control via Git with a GitHub remote ensured that all increments were tracked and reversible.

7.2 Key Implementation Details

7.2.1 Atomic Seat Booking

The join_ride_view implements a two-phase concurrent-safe booking algorithm. First, the target Ride record is retrieved with select_for_update(), which appends a FOR UPDATE clause to the SELECT, acquiring an InnoDB row-level exclusive lock for the duration of the transaction. This prevents any other concurrent transaction from reading or modifying the same ride row until the current transaction completes. Second, the seat decrement is performed as a SQL-level atomic operation: Ride.objects.filter(pk=ride.pk, seats_available__gt=0).update(seats_available=F('seats_available') - 1). The F-expression generates UPDATE rides_ride SET seats_available = seats_available - 1 WHERE id = %s AND seats_available > 0, ensuring the decrement only executes when seats are genuinely available, preventing negative seat counts even under extreme concurrency.

7.2.2 Multi-Stop Pickup Parsing

Drivers enter pickup stops as free text in a Textarea widget, separating stops with commas or newlines. The RideCreateForm.clean_pickup_stops_text() method normalizes this input by replacing all carriage returns and commas with newlines, splitting the result into individual stop strings, stripping whitespace, and filtering empty entries. In the form's save() override, the cleaned stop list is iterated with enumerate() to assign sequential stop_order values, and each unique stop is created as a RidePickupStop record linked to the new ride within the same database transaction.

7.2.3 Payment State Machine

Payment follows a deterministic two-state machine: PENDING → PAID. simulate_payment_view enforces two guards: (1) the parent ride's status must be FINALIZED — preventing premature payment on ongoing rides; (2) the participant's payment_status must be PENDING — preventing duplicate payment records. The state transition (payment_status = PAID, paid_at = timezone.now()) is persisted with a targeted save(update_fields=[...]) to avoid full-row updates, and the entire operation is wrapped in @transaction.atomic.

7.3 URL Routing

URL Pattern	View Function	Description
/	landing_view	Public landing page
/signup/	signup_view	User registration
/home/	home_view	Available rides listing
/rides/create/	create_ride_view	Ride creation form
/rides/current/	current_rides_view	Active rides dashboard
/rides/recent/	recent_rides_view	Completed ride history
/rides/<id>/join/	join_ride_view	POST: join a ride
/rides/<id>/finalize/	finalize_ride_view	POST: driver finalizes ride
/rides/<id>/pay/	simulate_payment_view	POST: passenger simulates payment
/accounts/login/	Django LoginView	Session-based login
/admin/	Django admin site	Admin CRUD panel

8. Testing

8.1 Testing Methodology

The project employs Django's built-in TestCase framework, which wraps each test method in a database transaction that is automatically rolled back after the test completes. This approach ensures full test isolation without requiring manual database cleanup or fixture teardown scripts. Django's test client (self.client) simulates HTTP requests without requiring a live server, enabling end-to-end testing of the view-form-model-database stack within a single Python process. Tests are located in rides/tests.py and are executed via python manage.py test.

8.2 Unit Test Cases

ID	Test Name	Scenario	Expected Outcome	Status
UT-01	test_passenger_can_join_ride	Authenticated passenger submits valid join form	seats_available decrements; status ACTIVE; RideParticipant created	PASS
UT-02	test_driver_can_finalize_ride	Driver posts finalize request for their own ride	status FINALIZED; finalized_at non-null	PASS
UT-03	test_passenger_can_simulate_payment	Passenger clicks pay after ride finalized	payment_status PAID; paid_at non-null	PASS
UT-04	test_driver_can_add_multiple_pickup_stops	Driver creates ride with comma/newline-separated stops	Correct ordered RidePickupStop records persisted	PASS

8.3 System Test Cases

ID	End-to-End Scenario	Expected Result
ST-01	Full ride lifecycle: Create → Browse → Join → Finalize → Pay	All state transitions complete; DB consistent at each step
ST-02	Two concurrent passengers attempt last available seat simultaneously	Exactly one booking succeeds; seats_available remains 0
ST-03	Ride with past departure_time submitted via Create Ride form	Form validation error displayed; no Ride record created
ST-04	375px mobile viewport: all pages rendered	No horizontal overflow; all UI elements accessible

9. Project Timeline

The following table presents the development timeline for UniCarpool across an eight-week development cycle:

Week	Phase	Activities	Status
Week 1	Planning & Requirements	Problem domain analysis, user story definition, initial ER sketching, technology selection	Done
Week 2	Database Design	Full ER diagram, normalization to BCNF, schema.sql authored, constraint and index design finalized	Done
Week 3	Django Setup & Auth	Project skeleton, config/, models.py initial version, migrations 0001, signup/login views and forms	Done
Week 4	Core Ride Features	RideCreateForm with pickup stop parsing, home_view with ORM optimization, join_ride_view with atomic transaction	Done
Week 5	Dashboard & Finalization	current_rides_view for driver and passenger roles, finalize_ride_view, migration 0002 for RidePickupStop	Done
Week 6	Payment & History	simulate_payment_view, recent_rides_view, pending payment count, full end-to-end lifecycle validation	Done
Week 7	Frontend & Testing	Custom dark-theme CSS, Bootstrap 5 responsive layout, automated test suite (rides/tests.py), manual cross-browser testing	Done
Week 8	Documentation	Project report (UniCarpool_Project_Report.docx), schema.sql, database-erd.md, README.md, proposal finalization	Done

10. Expected Results

10.1 System Deliverables

The completed UniCarpool system is expected to deliver a fully functional, tested, and documented ride-sharing web application. Core expected outputs include a working Django application accessible via a web browser, a MySQL database instantiated from the provided schema.sql with all constraints and indexes applied, a comprehensive automated test suite covering the critical workflow paths, and academic documentation suitable for university assessment.

10.2 Benefits to Users

- Cost reduction: Students sharing rides can reduce per-trip commuting costs by 50–75% compared to solo commercial ride-hailing, assuming typical Karachi route lengths and prevailing Careem/inDrive rates.
- Structured coordination: The platform replaces ad-hoc WhatsApp coordination with a structured, searchable, database-backed ride listing system with formal seat availability tracking.
- Safety improvement: Community-bounded registration ensures all platform users are verified members of the university community, improving trust compared to anonymous informal carpooling.
- Environmental benefit: Vehicle consolidation from even modest carpooling adoption measurably reduces per-capita CO2 emissions from daily student commutes.

10.3 Database Performance Expectations

With the implemented B-Tree indexes on departure_time, status, driver_id, user_id, and payment_status, common query patterns — particularly the home page ride listing (filtering by status, future departure, and excluding the current user's rides) and the payment history lookup — are expected to execute with index-range scans rather than full table scans, maintaining sub-100ms response times under realistic load. The atomic seat-booking implementation ensures that under concurrent load, database consistency is maintained without application-layer retry logic.

12. Future Enhancements

12.1 Scalability Improvements

The current single-server MySQL deployment is appropriate for development and small-scale institutional use. For production scalability, the following improvements are planned: implementation of a Redis-based caching layer for the home-page ride listing query (the highest-frequency read operation) using Django's cache framework; introduction of database read replicas to offload read-heavy query traffic (ride browsing, history pages) from the primary write server; and Gunicorn-based multi-worker deployment behind an Nginx reverse proxy for horizontal request handling.

12.2 Security Enhancements

- University email domain verification during signup, restricting registration to @nu.edu.pk addresses via regex validation in SignUpForm.
- Rate limiting on join and payment endpoints using django-ratelimit to prevent abuse and DoS-style booking floods.
- HTTPS enforcement in production via SECURE_SSL_REDIRECT = True and HSTS headers.
- Two-factor authentication (2FA) for driver finalization operations via django-otp.

12.3 Cloud Deployment

The application's environment-variable-driven configuration makes it immediately deployable to cloud platforms without code changes. Planned deployment targets include AWS EC2 with RDS for MySQL (providing managed automated backups and multi-AZ failover), Railway.app for zero-configuration PaaS deployment compatible with the existing requirements.txt, and Docker containerization with docker-compose.yml for CI/CD pipeline integration and reproducible environment packaging.

12.4 AI and Smart Matching

Route matching via the Google Maps Distance Matrix API would enable automatic suggestion of rides whose routes pass near a passenger's registered origin address. A demand forecasting module analyzing historical ride posting patterns could recommend optimal ride creation times. A machine learning model trained on route distance, departure time, and historical fare data could suggest fair per-seat pricing to drivers at ride creation time.

12.5 Feature Improvements

- Real-time seat availability updates using Django Channels (WebSockets) to reflect bookings without full page reload.
- In-app driver-passenger messaging post-booking using the Channels infrastructure.
- Star rating and review system for both drivers and passengers after ride completion, informing future booking decisions.
- Actual payment gateway integration via JazzCash, EasyPaisa, or Stripe, replacing the current payment simulation workflow.
- Push notifications via Firebase Cloud Messaging for ride status changes, new passenger joins, and payment confirmations.

- React Native or Flutter mobile application consuming a Django REST Framework (DRF) API backend for a native mobile experience.
- University ID verification at signup using the FAST-NUCES student portal API to restrict access to enrolled students and faculty.

13. Conclusion

UniCarpool was conceived as a practical response to a genuine and daily commuting challenge faced by students and faculty at FAST-NUCES Karachi, and was developed as a rigorous academic exercise in relational database design, web application security, and full-stack engineering. The project successfully delivers a functional, tested, and comprehensively documented ride-sharing platform covering the complete lifecycle of a shared university commute.

The relational database design at the heart of the system reflects careful and principled application of normalization theory, referential integrity constraint design, and indexing strategy. The four-table schema — built around rides_ride, rides_rideparticipant, rides_ridepickupstop, and Django's auth_user — accurately models the carpooling domain while maintaining BCNF compliance and cascading referential integrity throughout. The schema is documented in both Django migration files (for reproducibility) and a standalone schema.sql (for portability and academic review).

At the application layer, the project demonstrates mature use of the Django ORM including atomic transactions with row-level locking, N+1 query elimination via select_related and prefetch_related, and a stateful ride lifecycle management system enforced both at the ORM level and through database constraints. The custom form layer, multi-stop pickup parsing algorithm, and two-state payment machine all exemplify practical software engineering patterns applied thoughtfully within an academic project scope.

All stated functional objectives were accomplished: user authentication with PBKDF2-SHA256 hashing, ride creation with multi-stop pickup support, concurrent-safe seat booking, driver finalization, passenger payment simulation, and role-differentiated ride history display. Four automated unit tests provide a regression-ready test suite covering the critical happy paths, and manual integration testing confirmed correct behavior under concurrent booking conditions.

The project serves as a strong foundation for future development. With the planned additions of real geolocation route matching, live payment gateway integration, push notifications, and cloud deployment, UniCarpool could realistically evolve into a production-grade campus mobility tool serving the FAST-NUCES community. Its open-source architecture, environment-variable configuration, clean Django structure, and well-documented database schema ensure that such extensions can be pursued with minimal friction by future contributors.